

Resources and Deadlocks

1 Introduction

Central Idea

A **resource** is anything a process must **acquire**, **use**, and **release**. Examples include devices, files, database records, memory, and protected software objects.

Deadlock becomes possible when a process holds one exclusive resource while waiting for another resource held by a different process.

Main Point

A small change in resource-acquisition order can change a program from deadlock-free to deadlock-prone.

2 What Counts as a Resource?

A computer normally has many kinds of resources. Some resources have only one instance, while others have several identical instances.

Resource type	Example	Meaning of acquisition
Hardware device	Printer, Blu-ray drive, disk unit	A process gets permission to use the device.
Data object	Database record, shared file	A process gets exclusive access before reading or updating it.
Memory	Main-memory pages or address space	A process occupies memory while it runs.
Software object	Lock, semaphore-protected object	A process enters a protected region.

Identical Instances

If several identical copies exist, any one copy can satisfy the request. For example, if three Blu-ray drives are free, a process requesting one drive can receive any of them.

3 Preemptable and Nonpreemptable Resources

Preemptable Resource

A **preemptable resource** can be taken away from its current owner without damaging the computation or causing failure.

Example: memory on a system with paging or swapping. The operating system can swap a process out, give memory to another process, and later restore the swapped-out process.

Nonpreemptable Resource

A **nonpreemptable resource** cannot be taken away from its current owner at an arbitrary moment without risking failure or corrupted output.

Example: a Blu-ray recorder while writing. Taking it away in the middle of writing may corrupt the disc.

Kind	Can be taken away?	Typical deadlock effect
Preemptable	Yes, if the system can save and restore state safely.	Potential deadlocks can often be resolved by reallocating the resource.
Nonpreemptable	No, not safely at an arbitrary time.	Real deadlocks usually involve these resources.

Context Matters

Preemptability depends on the system. Memory is preemptable on many PCs with paging, but may not be preemptable on a device without paging or swapping.

4 Standard Resource-Use Sequence

Resource Life Cycle

Request resource → Use resource → Release resource

If the resource is free, the process gets it. If it is busy, the process waits.

- Some systems block the process automatically.
- Some systems return an error, so the process sleeps and retries.
- A process that only retries cannot do useful work, so it is effectively blocked.

Simplifying assumption

For deadlock discussion, we usually assume that a process whose resource request is denied is put to sleep until the resource becomes available.

5 Resource Acquisition with Semaphores

When the operating system does not directly manage a resource, processes can protect it with a semaphore or mutex.

Semaphore Protection

For a binary semaphore initialized to 1, **down** acquires the resource and **up** releases it.

5.1 One Resource

```
typedef int semaphore;
semaphore resource_1 = 1;

void process_A(void) {
    down(&resource_1);    /* request resource 1 */
    use_resource_1();      /* use resource 1 */
    up(&resource_1);      /* release resource 1 */
}
```

Only one process can use `resource_1` at a time.

5.2 Two Resources

```
typedef int semaphore;
semaphore resource_1 = 1;
semaphore resource_2 = 1;

void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources();
    up(&resource_2);
    up(&resource_1);
}
```

Sequential acquisition is common. It becomes risky when different processes acquire the same resources in different orders.

6 Deadlock-Free Acquisition Order

Ordering Rule

If every process acquires shared resources in the same global order, circular waiting is avoided for those resources.

Both processes below request `resource_1` before `resource_2`. One process may wait, but no circular wait forms.

```
typedef int semaphore;
semaphore resource_1 = 1;
semaphore resource_2 = 1;

void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources();
    up(&resource_2);
    up(&resource_1);
}

void process_B(void) {
    down(&resource_1);
    down(&resource_2);
}
```

```
    use_both_resources();  
    up(&resource_2);  
    up(&resource_1);  
}
```

What happens at runtime?

If *A* gets `resource_1` first, *B* blocks before holding anything. *A* can get `resource_2`, finish, release both, and then *B* can run.

7 Potential Deadlock from Different Orders

Here *A* requests `resource_1` first, but *B* requests `resource_2` first.

```
typedef int semaphore;  
semaphore resource_1 = 1;  
semaphore resource_2 = 1;  
  
void process_A(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources();  
    up(&resource_2);  
    up(&resource_1);  
}  
  
void process_B(void) {  
    down(&resource_2);  
    down(&resource_1);  
    use_both_resources();  
    up(&resource_1);  
    up(&resource_2);  
}
```

Potential deadlock

This code does not deadlock on every run. It deadlocks only if *A* holds `resource_1` while *B* holds `resource_2`.

8 Worked Deadlock Trace

Problem

Processes *A* and *B* both need resources R_1 and R_2 . Process *A* requests R_1 then R_2 . Process *B* requests R_2 then R_1 . Show how deadlock can occur.

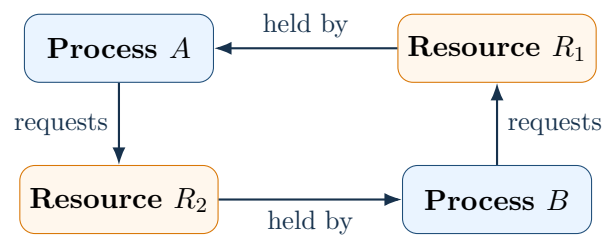
Solution

Step	Process A	Process B	State
1	Acquires R_1 .	Not yet running.	A holds R_1 .
2	Pauses before requesting R_2 .	Acquires R_2 .	A holds R_1 , B holds R_2 .
3	Requests R_2 .	Still holds R_2 .	A blocks because R_2 is unavailable.
4	Still blocked.	Requests R_1 .	B blocks because R_1 is unavailable.
5	Waiting for B to release R_2 .	Waiting for A to release R_1 .	Neither process can continue.

Answer

The system is deadlocked: A waits for B , and B waits for A . Neither can reach the release code.

9 Resource Allocation View



Circular Wait Pattern

A waits for R_2 , R_2 is held by B , B waits for R_1 , R_1 is held by A

This cycle is the deadlock. Because the resources are nonpreemptable, the operating system cannot simply take them away safely.

Introduction to Deadlocks

1 Deadlock Definition

Formal Definition

A set of processes is **deadlocked** if every process in the set is waiting for an event that only another process in the same set can cause.

Since all processes in the set are waiting, none of them can run and none of them can cause the event needed to wake another process. Therefore, the waiting continues forever.

Resource Deadlock

In the most common case, each process is waiting for a resource held by another deadlocked process. This is called a **resource deadlock**.

Model Assumptions

The basic model assumes single-threaded processes and no interrupts that can wake a blocked process. Otherwise, an alarm or interrupt could break the deadlock by waking a process.

2 Conditions for Resource Deadlock

Coffman Conditions

A resource deadlock is possible only when all four conditions below hold at the same time.

Condition	Meaning
Mutual exclusion	Each resource is either available or assigned to exactly one process.
Hold and wait	A process holding one resource is allowed to request another resource.
No preemption	A granted resource cannot be forcibly taken away; it must be released by its owner.
Circular wait	Two or more processes form a cycle, where each waits for a resource held by the next process.

Key Rule

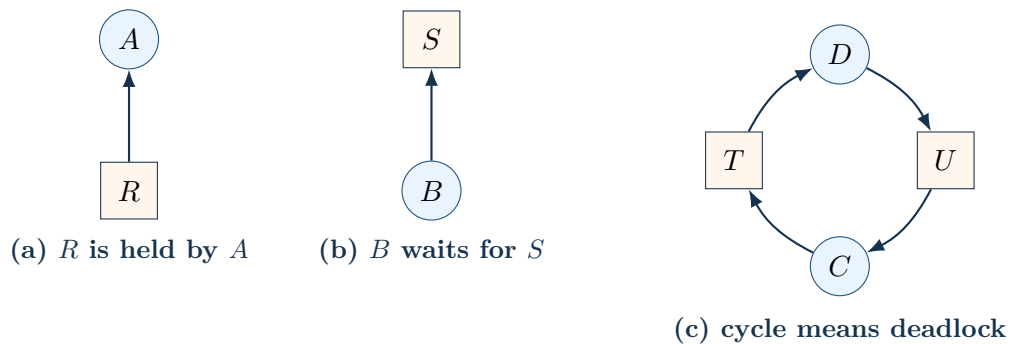
If any one Coffman condition is absent, resource deadlock cannot occur.

3 Resource-Allocation Graphs

Deadlocks can be modeled with a directed graph.

- A **process** is shown as a circle.

- A **resource** is shown as a square.
- A directed edge shows either ownership or waiting.



Graph edge	Meaning
Resource $\rightarrow$ Process	The resource has been granted and is currently held by that process.
Process $\rightarrow$ Resource	The process is blocked waiting for that resource.
Cycle	Deadlock exists, assuming one instance of each resource type.

4 Deadlock Cycle Example

Setup

Process C holds resource U and requests resource T . Process D holds resource T and requests resource U .

$$C \rightarrow T \rightarrow D \rightarrow U \rightarrow C$$

Interpretation

C cannot continue until D releases T . D cannot continue until C releases U . Neither process can reach its release step, so both wait forever.

5 Modeling a Request Sequence

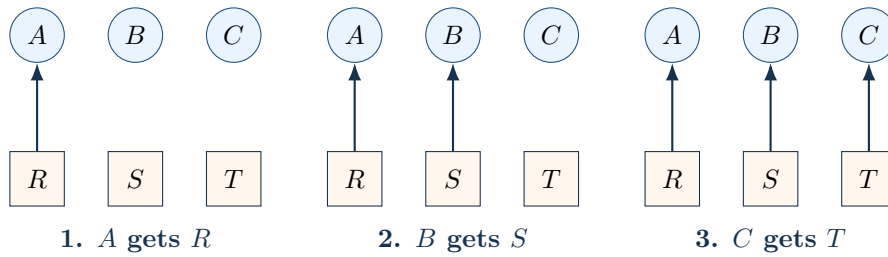
Resource graphs can be built step by step from process requests and releases. After each step, check whether the graph contains a cycle.

Processes and Resources

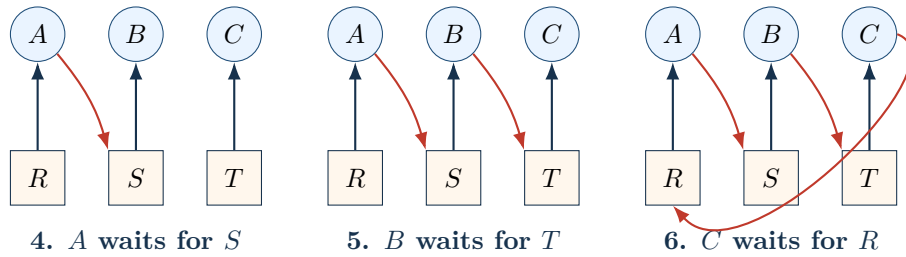
There are three processes A , B , and C , and three resources R , S , and T . Each resource has one instance.

Process	Request and release order
A	Request R , request S , release R , release S .
B	Request S , request T , release S , release T .
C	Request T , request R , release T , release R .

Deadlocking request order: first three grants

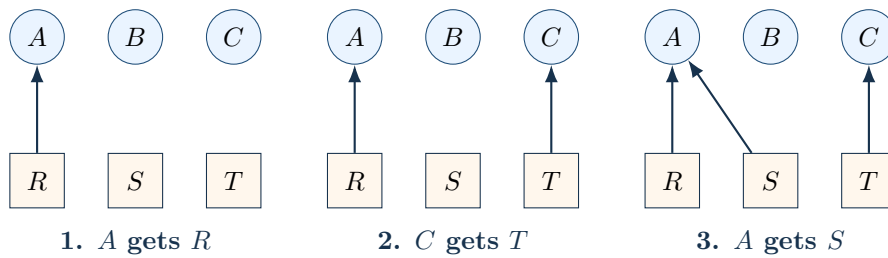


Deadlocking request order: cycle appears

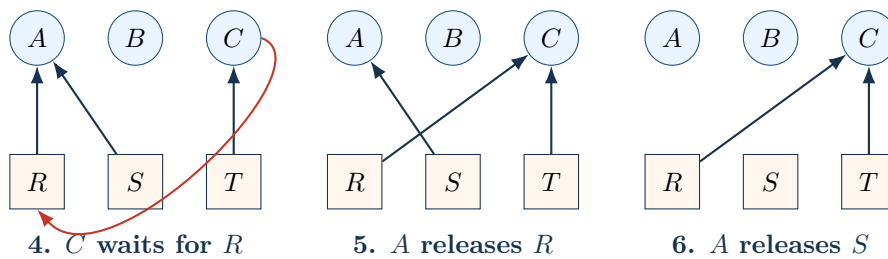


At step 6, the graph has the cycle $A \rightarrow S \rightarrow B \rightarrow T \rightarrow C \rightarrow R \rightarrow A$.

Safer request order: no cycle



Safer request order: waiting is temporary



There is waiting in step 4, but there is no cycle. Once A releases R, C can continue.

5.1 Deadlocking Schedule

Step	Request	Result
1	<i>A</i> requests <i>R</i> .	<i>A</i> holds <i>R</i> .
2	<i>B</i> requests <i>S</i> .	<i>B</i> holds <i>S</i> .
3	<i>C</i> requests <i>T</i> .	<i>C</i> holds <i>T</i> .
4	<i>A</i> requests <i>S</i> .	<i>A</i> blocks; <i>S</i> is held by <i>B</i> .
5	<i>B</i> requests <i>T</i> .	<i>B</i> blocks; <i>T</i> is held by <i>C</i> .
6	<i>C</i> requests <i>R</i> .	<i>C</i> blocks; <i>R</i> is held by <i>A</i> .

Cycle Formed

$$A \rightarrow S \rightarrow B \rightarrow T \rightarrow C \rightarrow R \rightarrow A$$

6 Avoiding the Bad Schedule

The operating system is not forced to grant every request immediately. If granting a request may lead to deadlock, the system can delay that process and run other processes first.

Step	Safer order	Result
1	<i>A</i> requests <i>R</i> .	<i>A</i> holds <i>R</i> .
2	<i>C</i> requests <i>T</i> .	<i>C</i> holds <i>T</i> .
3	<i>A</i> requests <i>S</i> .	<i>A</i> holds <i>R</i> and <i>S</i> .
4	<i>C</i> requests <i>R</i> .	<i>C</i> waits for <i>A</i> .
5	<i>A</i> releases <i>R</i> .	<i>C</i> can later get <i>R</i> .
6	<i>A</i> releases <i>S</i> .	<i>B</i> can safely receive <i>S</i> later.

Main Use of Graphs

Resource-allocation graphs help test a request/release sequence: apply each step, update the graph, and check for cycles.

7 Strategies for Handling Deadlocks

Strategy	Basic idea
Ignore the problem	Assume deadlocks are rare and do nothing special.
Detection and recovery	Allow deadlocks, detect them, then recover by taking action.
Dynamic avoidance	Decide carefully whether each resource request is safe before granting it.
Prevention	Design the system so at least one Coffman condition cannot hold.

Deadlock Detection

1 Main Idea

Deadlock Detection

Deadlock detection means allowing deadlocks to occur, then checking the system state to find whether a deadlock exists and which processes are involved.

The detection method depends on the resource model:

- If there is **one resource of each type**, use a resource-allocation graph and look for cycles.
- If there are **multiple resources of each type**, use a matrix-based detection algorithm.

2 One Resource of Each Type

Key Rule

When there is only one instance of each resource type, a cycle in the resource-allocation graph means deadlock.

In the graph:

- Process nodes are circles.
- Resource nodes are squares.
- Resource $\rightarrow$ process means the process holds the resource.
- Process $\rightarrow$ resource means the process is waiting for the resource.

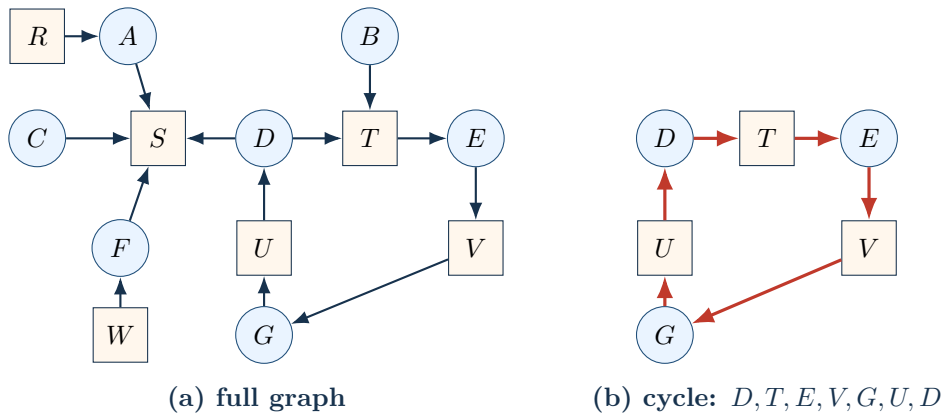
3 Resource Graph Example

System State

There are seven processes *A* through *G*, and six resources *R* through *W*.

Process	Holds and requests
<i>A</i>	Holds <i>R</i> , wants <i>S</i> .
<i>B</i>	Holds nothing, wants <i>T</i> .
<i>C</i>	Holds nothing, wants <i>S</i> .
<i>D</i>	Holds <i>U</i> , wants <i>S</i> and <i>T</i> .
<i>E</i>	Holds <i>T</i> , wants <i>V</i> .
<i>F</i>	Holds <i>W</i> , wants <i>S</i> .
<i>G</i>	Holds <i>V</i> , wants <i>U</i> .

Resource Graph and Extracted Cycle



Conclusion

Processes D , E , and G are deadlocked because they are part of the cycle. Processes A , C , and F are waiting for S , but they are not deadlocked.

4 Cycle Detection Algorithm

For real systems, visual inspection is not enough. The graph can be searched using depth-first search.

Cycle Detection for One Resource of Each Type

```

for each node N in the graph:
    L = empty list
    mark all arcs as uninspected
    current = N

    repeat:
        append current to L
        if current appears twice in L:
            report cycle and stop

        if current has an unmarked outgoing arc:
            mark one such arc
            current = node reached by that arc
        else if current is N:
            stop this search; no cycle from N
        else:
            remove current from L
            current = previous node in L

    if all starting nodes are checked:
        report no deadlock
    
```

How it works

The algorithm starts from each node and performs depth-first search. If the search reaches a node already in the current path L , a cycle has been found.

5 Multiple Resources of Each Type

When there are multiple instances of a resource type, a simple cycle test is not enough. Instead, use vectors and matrices.

Data Structures for Matrix Detection

Existing resources

$$E = (E_1, E_2, \dots, E_m)$$

Available resources

$$A = (A_1, A_2, \dots, A_m)$$

Current allocation matrix C

$$C = \begin{bmatrix} C_{11} & C_{12} & \cdots & C_{1m} \\ C_{21} & C_{22} & \cdots & C_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ C_{n1} & C_{n2} & \cdots & C_{nm} \end{bmatrix}$$

Row i : resources currently held by process P_i .

Request matrix R

$$R = \begin{bmatrix} R_{11} & R_{12} & \cdots & R_{1m} \\ R_{21} & R_{22} & \cdots & R_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ R_{n1} & R_{n2} & \cdots & R_{nm} \end{bmatrix}$$

Row i : additional resources requested by process P_i .

Symbol	Meaning
E	Existing resource vector. E_j is the total number of resources of class j .
A	Available resource vector. A_j is the number of currently free resources of class j .
C	Current allocation matrix. C_{ij} is how many resources of class j process P_i holds.
R	Request matrix. R_{ij} is how many more resources of class j process P_i wants.

Resource Accounting Invariant

For every resource class j ,

$$\sum_{i=1}^n C_{ij} + A_j = E_j$$

All resources are either allocated to some process or available.

6 Vector Comparison

For two resource vectors X and Y , write $X \leq Y$ if every component of X is less than or equal to the corresponding component of Y .

$$X \leq Y \iff X_j \leq Y_j \text{ for every } 1 \leq j \leq m$$

This means process P_i 's request can be satisfied if row $R_i \leq A$.

7 Matrix Detection Algorithm

Detection with Multiple Resource Instances

```
mark every process as unmarked

repeat:
    find an unmarked process  $P_i$  such that  $R_i \leq A$ 

    if such a process exists:
         $A = A + C_i$ 
        mark  $P_i$ 
    else:
        stop

all unmarked processes are deadlocked
```

Meaning

If $R_i \leq A$, process P_i can finish using currently available resources. After it finishes, it releases its current allocation C_i , so A increases.

8 Worked Matrix Example

Setup

There are three processes and four resource classes: tape drives, plotters, scanners, and Blu-ray drives.

$$E = (4, 2, 3, 1), \quad A = (2, 1, 0, 0)$$

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix}, \quad R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}$$

Step 1: Find a process that can finish

$$R_1 = (2, 0, 0, 1) \not\leq A = (2, 1, 0, 0)$$

Process 1 cannot finish because it needs one Blu-ray drive, but none is available.

$$R_2 = (1, 0, 1, 0) \not\leq A = (2, 1, 0, 0)$$

Process 2 cannot finish because it needs one scanner, but none is available.

$$R_3 = (2, 1, 0, 0) \leq A = (2, 1, 0, 0)$$

Process 3 can finish. Add its allocation $C_3 = (0, 1, 2, 0)$ to A :

$$A = (2, 1, 0, 0) + (0, 1, 2, 0) = (2, 2, 2, 0)$$

Step 2: Process 2 can finish

$$R_2 = (1, 0, 1, 0) \leq A = (2, 2, 2, 0)$$

Add $C_2 = (2, 0, 0, 1)$ to A :

$$A = (2, 2, 2, 0) + (2, 0, 0, 1) = (4, 2, 2, 1)$$

Step 3: Process 1 can finish

$$R_1 = (2, 0, 0, 1) \leq A = (4, 2, 2, 1)$$

All processes can be marked, so there is no deadlock.

Answer

The system is not deadlocked. A possible completion order is $P_3 \rightarrow P_2 \rightarrow P_1$.

9 When to Run Detection

Detection time	Tradeoff
After every resource request	Finds deadlocks early, but uses more CPU time.
Every k minutes	Reduces overhead, but deadlocks may persist longer.
When CPU utilization is low	Useful because many deadlocked processes leave few runnable processes.

The Ostrich Algorithm

1 Main Idea

Definition

The **Ostrich algorithm** is a deadlock-handling strategy in which the operating system simply ignores the deadlock problem.

The name is based on the idea of hiding one's head in the sand and pretending the problem does not exist. In operating systems, this means no special deadlock detection, avoidance, or prevention is performed.

Core Point

The Ostrich algorithm is not a real algorithm for solving deadlocks. It is a policy choice: ignore rare deadlocks because solving them may cost more than tolerating them.

2 Why Ignore Deadlocks?

Deadlock handling can be expensive. Detection needs runtime checks, avoidance needs information about future resource needs, and prevention can reduce flexibility or performance.

Reason	Explanation
Deadlocks may be rare	If deadlocks happen very infrequently, the cost of preventing every one may not be justified.
Handling has overhead	Detection and avoidance algorithms consume CPU time, memory, and bookkeeping effort.
Prevention restricts programs	Forcing strict rules, such as fixed resource ordering, can make systems less convenient or less efficient.
Recovery may be simple	A user or administrator can kill and restart a stuck program when the problem occurs.

3 When It Makes Sense

Practical Use

The Ostrich algorithm is reasonable when deadlocks are rare, the damage is small, and the cost of full deadlock handling is high.

Many general-purpose operating systems use this practical attitude for some kinds of deadlocks. They do not try to prevent every possible deadlock in user programs because doing so would make the system more complex and slower for normal cases.

Example

If two user programs occasionally deadlock while using files or locks, the system may simply let the user terminate one of them. This is often cheaper than constantly tracking every possible deadlock.

4 When It Is Not Acceptable

Do not use blindly

Ignoring deadlocks is dangerous when failure is frequent, costly, or safety-critical.

System type	Why ignoring is bad
Real-time control system	A deadlock may miss a deadline and cause physical damage.
Banking or transaction system	A deadlock can block important operations and leave users waiting.
Database server	Deadlocks may be common enough that detection and rollback are necessary.
Medical or aviation system	Manual restart is not an acceptable recovery plan.

5 Comparison with Other Strategies

Strategy	Main idea	Cost
Ostrich algorithm	Ignore the problem.	Lowest overhead, but no protection.
Detection and recovery	Let deadlocks occur, detect them, then recover.	Runtime checks and recovery work.
Avoidance	Grant a request only if the resulting state is safe.	Needs advance information and decision logic.
Prevention	Break at least one Coffman condition structurally.	Can restrict resource use and reduce flexibility.

6 Simple Decision Rule

Use or Avoid

Use Ostrich	if deadlocks are rare and cheap to recover from.
Do not use Ostrich	if deadlocks are frequent, costly, or safety-critical.

7 Worked Example

Problem

A desktop operating system has a rare deadlock in a user application once every few months. Preventing it would require expensive resource tracking for every program. Should the system use the Ostrich algorithm?

Solution

The deadlock is rare and affects only a user application. The user can terminate and restart the application. Constant deadlock prevention would add overhead to every normal execution.

Answer

Using the Ostrich algorithm is reasonable here because the cost of full prevention is greater than the cost of occasional manual recovery.

8 Common Mistakes

Mistake 1

Do not say the Ostrich algorithm prevents deadlocks. It does not prevent, detect, or recover from them automatically.

Mistake 2

Do not assume ignoring deadlocks is always careless. It can be a rational engineering choice when deadlocks are extremely rare.

Mistake 3

Do not use this strategy for systems where a deadlock can cause serious loss, missed deadlines, or unsafe behavior.

Recovery from Deadlock

1 Main Idea

Deadlock Recovery

Deadlock recovery is the action taken after a deadlock has already been detected, so that the system can make progress again.

Detection only tells us that a deadlock exists. Recovery decides how to break it. The common methods are:

- recover through preemption,
- recover through rollback,
- recover by killing one or more processes.

Important

Deadlock recovery methods are usually unpleasant. They may lose work, require manual intervention, or terminate processes.

2 Recovery through Preemption

Preemption

Recovery through preemption means temporarily taking a resource away from one process and giving it to another process so the deadlock can be broken.

This works only when the resource can be removed without corrupting the process or the resource state.

Example: Printer

An operator may pause a process using a printer, collect the printed pages, suspend the process, give the printer to another process, and later restart the original process.

Issue	Explanation
Resource-dependent	Some resources can be taken away safely, but many cannot.
May need manual help	Batch systems may require an operator to intervene.
Victim choice matters	Choose a process whose resources can be taken back most easily.
Often difficult	The process should ideally not notice that the resource was removed.

Limitation

Preemption is often impossible for nonpreemptable resources. For example, taking away a device in the middle of an irreversible operation may corrupt output.

3 Recovery through Rollback

Checkpoint

A **checkpoint** is a saved copy of a process state, including its memory image and the resources assigned to it.

If deadlocks are expected, the system can periodically checkpoint processes. When a deadlock is detected, a process holding a needed resource can be rolled back to an earlier checkpoint before it acquired that resource.

Rollback Idea

rollback process → release needed resource → restart from checkpoint

Why old checkpoints are kept

New checkpoints should not overwrite old ones. A sequence of checkpoints gives the system several possible rollback points.

Cost of rollback

All work done after the checkpoint is lost. Any external output produced after the checkpoint may need to be discarded because it may be produced again after restart.

4 Recovery by Killing Processes

Process Killing

The simplest recovery method is to terminate one or more processes until the deadlock is broken.

There are two main choices:

- kill a process that is inside the deadlock cycle,
- kill a process outside the cycle if it holds resources needed by deadlocked processes.

Victim Selection

Where possible, kill a process that can be safely restarted from the beginning with no harmful side effects.

5 Safe and Unsafe Victims

Process type	Safe to kill?	Reason
Compilation	Usually safe	It reads source files and produces an object file. It can be rerun.
Printing job	Sometimes unsafe	Printed output may need to be discarded or restarted carefully.
Database update	Often unsafe	Re-running may apply the update twice and corrupt data.
Transaction with roll-back support	Safer	The system can undo partial work before retrying.

Database example

If a process adds 1 to a database field, killing it and running it again may add 1 twice. That changes the data incorrectly.

6 Comparison of Recovery Methods

Method	Main idea	Main problem
Preemption	Take a resource from one process and give it to another.	Many resources cannot be safely preempted.
Rollback	Restart a process from an earlier checkpoint.	Work after the checkpoint is lost.
Killing processes	Terminate one or more processes to release resources.	May lose work or cause incorrect external effects.

7 Worked Example

Problem

Two processes are deadlocked. Process P_1 holds a printer and wants a plotter. Process P_2 holds a plotter and wants a printer. A third process P_3 holds another printer and another plotter but is not deadlocked. How can killing P_3 help?

Solution

Although P_3 is not part of the deadlock cycle, it holds resources that the deadlocked processes need. Killing P_3 releases its printer and plotter. Then one of the deadlocked processes may receive the released resource and continue.

Answer

A recovery victim does not always have to be inside the deadlock cycle. It can be outside the cycle if killing it releases resources needed by the cycle.

8 Choosing a Victim

Good victim

A good victim is cheap to stop, easy to restart, and unlikely to leave corrupted data or duplicated external effects.

Typical factors:

- How much work will be lost?
- Can the process be restarted safely?
- What resources will be released?
- Is the process interactive or batch?
- Could killing it corrupt files, databases, or output?

Resource Trajectories

1 Main Idea

Resource Trajectory

A **resource trajectory** is a graph that shows the combined progress of two processes and helps visualize safe and unsafe states in deadlock avoidance.

The graphical method is mainly for understanding. It does not directly become a practical operating-system algorithm, but it explains why some resource requests must be delayed.

Connection to Deadlock Avoidance

Deadlock avoidance is based on refusing a resource request if granting it would move the system into an unsafe state.

2 The Two-Process Model

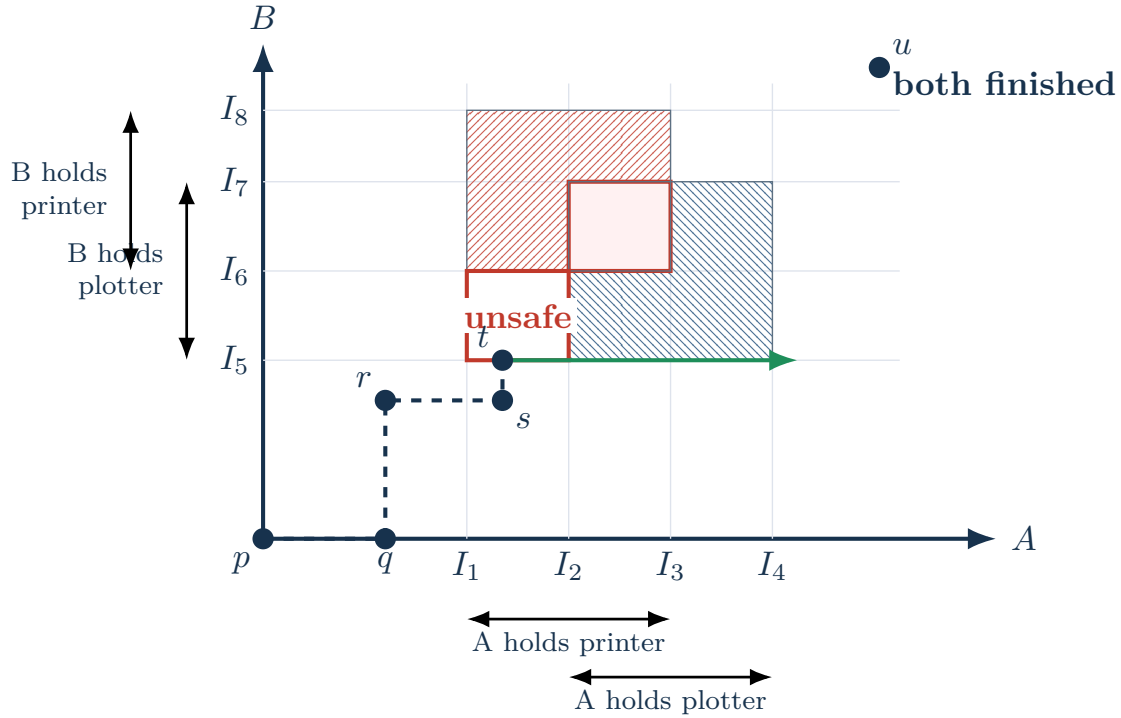
Assume two processes, A and B , and two resources: a printer and a plotter.

- The horizontal axis shows how many instructions process A has executed.
- The vertical axis shows how many instructions process B has executed.
- Each point in the graph is a joint state of both processes.
- On a single processor, movement is horizontal or vertical, not diagonal.

Direction of movement

The path can move only east or north. Processes move forward in time, so the trajectory cannot move west or south.

3 Resource Trajectory Diagram



Marking	Meaning
	Forbidden region: both processes would hold the printer.
	Forbidden region: both processes would hold the plotter.
	Unsafe region: entering it can lead to deadlock.
	Safe choice at t : run A until it releases the plotter.

Figure: Resource trajectory for two processes and two resources.

4 Meaning of the Lines

Point or line	Meaning
p	Initial state: neither process has executed instructions.
q	Process A has run; process B has not run yet.
I_1	A requests the printer.
I_2	A requests the plotter.
I_3	A releases the printer.
I_4	A releases the plotter.
I_5	B requests the plotter.
I_6	B requests the printer.
I_7	B releases the plotter.
I_8	B releases the printer.
u	Both processes have finished.

5 Forbidden Regions

Forbidden Region

A **forbidden region** is a part of the graph that cannot be entered because it would mean both processes hold the same exclusive resource at the same time.

- One shaded region means both processes would have the printer.
- The other shaded region means both processes would have the plotter.
- Mutual exclusion makes both regions impossible.

6 Unsafe Region

Unsafe Region

An **unsafe region** is a state region from which the system may be forced into deadlock if scheduling continues badly.

In the diagram, the unsafe box is bounded by:

$$I_1 < A < I_2 \quad \text{and} \quad I_5 < B < I_6$$

Inside this region:

- A already holds the printer and will soon request the plotter.
- B already holds the plotter and will soon request the printer.
- At the intersection of I_2 and I_6 , both processes wait for each other.

Deadlock point

At $I_2 \cap I_6$, process A asks for the plotter while process B asks for the printer. Each resource is already held by the other process.

7 Scheduler Decision at t

At point t , process B requests the plotter. The operating system must decide whether to grant it.

Decision	Result
Grant B 's request	The system enters the unsafe region and may deadlock later.
Suspend B	A can continue until it releases the plotter, keeping the system safe.

Correct action

At point t , the safe action is to run process A until it passes I_4 , where A has released the plotter.

8 Important Observations

- A safe state is not necessarily deadlock-free forever; it means there exists at least one safe path to completion.
- An unsafe state is not deadlock yet, but it can lead to deadlock.
- Avoidance algorithms prevent entry into unsafe states.
- The trajectory diagram gives intuition for safe states, but real systems need algorithmic checks.

Safe and Unsafe States

1 Main Idea

System State

In deadlock avoidance, the current state is described by the resource data structures E , A , C , and R : existing resources, available resources, current allocation, and remaining requests.

A deadlock-avoidance algorithm checks whether granting a resource request keeps the system in a safe state.

Safe State

A state is **safe** if there exists at least one scheduling order in which every process can run to completion, even if all processes immediately request their maximum remaining resources.

Unsafe State

A state is **unsafe** if the system cannot guarantee that all processes can finish. An unsafe state is not necessarily deadlocked, but it may lead to deadlock.

2 Safe Sequence

Definition

A **safe sequence** is an order of processes $P_1, P_2, \dots, P_n$ such that each process can receive its remaining needed resources, finish, and release its resources for the next process.

Test for One Resource Type

For a process P_i ,

$$\text{Need}_i = \text{Max}_i - \text{Has}_i$$

$$P_i \text{ can finish if } \text{Need}_i \leq \text{Free}$$

3 Example 1: A Safe State

Initial State

There are 10 total instances of one resource. Processes A , B , and C currently hold some instances and may need more later.

Process	Has	Max	Need = Max - Has
A	3	9	6
B	2	4	2
C	2	7	5

$$\text{Allocated} = 3 + 2 + 2 = 7, \quad \text{Free} = 10 - 7 = 3$$

Since B 's need is 2, and $2 \leq 3$, process B can safely finish first.

4 Safe-State Trace

Step	Free	Action	New free
(a)	3	Run B . It needs 2 more, so it can reach its max 4.	1 while B runs
(b)	1	B finishes and releases all 4 resources.	$1 + 4 = 5$
(c)	5	Run C . It needs 5 more, so it can reach its max 7.	0 while C runs
(d)	0	C finishes and releases all 7 resources.	$0 + 7 = 7$
(e)	7	Run A . It needs 6 more, so it can finish.	all processes finish

Safe Sequence

The state is safe because the sequence

$$B \rightarrow C \rightarrow A$$

allows every process to finish.

5 Visual Trace of the Safe State

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3

(a)

	Has	Max
A	3	9
B	4	4
C	2	7

Free: 1

(b)

	Has	Max
A	3	9
B	0	–
C	2	7

Free: 5

(c)

	Has	Max
A	3	9
B	0	–
C	7	7

Free: 0

(d)

	Has	Max
A	3	9
B	0	–
C	0	–

Free: 7

(e)

Transition	Meaning
(a) $\rightarrow$ (b)	B receives 2 more resources and reaches its maximum of 4.
(b) $\rightarrow$ (c)	B finishes, so its 4 resources return to the free pool.
(c) $\rightarrow$ (d)	C receives 5 more resources and reaches its maximum of 7.
(d) $\rightarrow$ (e)	C finishes, so A can later receive what it needs.

Figure: Safe sequence $B \rightarrow C \rightarrow A$. A dash means the process has finished.

6 Example 2: An Unsafe State

Now suppose process A requests and receives one more resource.

9 Unsafe Does Not Mean Deadlocked

Important distinction

An unsafe state is not the same as a deadlocked state.

Starting from the unsafe state, the system may still run for a while. For example, B can complete. It is also possible that A might release a resource before asking for more, allowing C to finish. However, the system cannot guarantee completion under worst-case requests.

State type	Meaning
Safe	There exists a guaranteed order in which all processes can finish.
Unsafe	No guaranteed completion order exists, but deadlock may not have happened yet.
Deadlocked	Processes are already stuck waiting for events only each other can cause.

The Banker's Algorithm

1 Main Idea

Banker's Algorithm

The **Banker's Algorithm** is a deadlock-avoidance algorithm. Before granting a resource request, it checks whether the resulting state is safe.

The algorithm was introduced by Dijkstra. It is based on the idea of a banker who gives loans only when the bank can still satisfy all customers in some safe order.

Decision Rule

Grant a request only if granting it keeps the system in a safe state. If the request would create an unsafe state, postpone it.

2 Banking Analogy

Banking term	Operating-system meaning
Customer	Process.
Credit unit	Resource instance, such as a tape drive or printer.
Maximum credit line	Maximum number of resources the process may need.
Current loan	Resources currently allocated to the process.
Banker	Operating system.

3 Single-Resource Version

Assume there is only one type of resource. Each process declares its maximum possible need before running.

Single-Resource Need

$$\text{Need}_i = \text{Max}_i - \text{Has}_i$$

A state is safe if there is some order in which each process can receive its remaining need, finish, and release its resources.

4 Single-Resource Example

	Has	Max
A	0	6
B	0	5
C	0	4
D	0	7
Free: 10		

	Has	Max
A	1	6
B	1	5
C	2	4
D	4	7
Free: 2		

	Has	Max
A	1	6
B	2	5
C	2	4
D	4	7
Free: 1		

(a) initial safe state (b) safe state (c) unsafe state

Figure: Three single-resource allocation states. State (c) is unsafe.

Why state (b) is safe

In state (b), the banker has 2 free units. Process *C* needs only 2 more units, so *C* can finish and release all 4 units it holds. Then another process can finish, and eventually all can complete.

Why state (c) is unsafe

If *B*'s request for one more unit is granted, only 1 unit remains free. No process can be guaranteed to receive enough resources to finish if all processes request their maximum needs.

5 Single-Resource Algorithm

Banker's Algorithm: One Resource Type

```
when process  $P_i$  requests  $k$  resources:
    if  $k > \text{Need}_i$ :
        reject the request

    if  $k > \text{Free}$ :
        make  $P_i$  wait

    pretend to grant the request:
         $\text{Has}_i = \text{Has}_i + k$ 
         $\text{Free} = \text{Free} - k$ 

    if the resulting state is safe:
        grant the request
    else:
        undo the pretend allocation
        make  $P_i$  wait
```

6 Testing Safety

Safety Test

```
Work = Free
mark all processes unfinished

repeat:
    find an unfinished process  $P_i$  with  $\text{Need}_i \leq \text{Work}$ 

    if such a process exists:
        Work = Work + Hasi
        mark  $P_i$  finished
    else:
        stop
```

```

if all processes are marked finished:
    the state is safe
else:
    the state is unsafe

```

Meaning

The safety test asks: “Can some process finish with the currently available resources? If yes, assume it finishes and returns its resources. Repeat.”

7 Multiple-Resource Version

For multiple resource types, use the same idea with vectors and matrices.

Symbol	Meaning
E	Existing resource vector: total resources of each type.
P	Possessed resource vector: total resources currently allocated.
A	Available resource vector: resources currently free.
C	Current allocation matrix: resources currently assigned to each process.
R	Remaining request matrix: resources each process still needs to finish.

Vector Relationship

$$A = E - P$$

8 Multiple-Resource Example

The resource types are tape drives, plotters, printers, and Blu-ray drives.

Resources assigned C					Resources still needed R				
	Tape	Plotter	Printer	Blu-ray		Tape	Plotter	Printer	Blu-ray
A	3	0	1	1	A	1	1	0	0
B	0	1	0	0	B	0	1	1	2
C	1	1	1	0	C	3	1	0	0
D	1	1	0	1	D	0	0	1	0
E	0	0	0	0	E	2	1	1	0

$$E = (6, 3, 4, 2), \quad P = (5, 3, 2, 2), \quad A = (1, 0, 2, 0)$$

9 Multiple-Resource Safety Algorithm

Banker's Algorithm: Multiple Resources

```
Work = A
mark all processes unfinished

repeat:
    find an unfinished process  $P_i$  such that  $R_i \leq \text{Work}$ 

    if such a process exists:
        Work = Work +  $C_i$ 
        mark  $P_i$  finished
    else:
        stop

if every process is marked finished:
    the state is safe
else:
    the state is unsafe
```

Vector comparison

$R_i \leq \text{Work}$ means every resource need in row R_i is less than or equal to the corresponding available amount in Work .

10 Example Decision

Request from B

Suppose process B requests one printer. The system pretends to grant the request and then checks whether the resulting state is still safe.

This request can be granted if some process can still finish first after the pretend allocation. In the example, process D can finish, returning its resources. Then other processes can finish afterward.

Request from E

After giving B one printer, suppose E requests the last remaining printer.

Granting E 's request would leave:

$$A = (1, 0, 0, 0)$$

No process can be guaranteed to finish from that state, so E 's request must be postponed.

11 Why the Algorithm Is Rarely Used

Practical limitation

The Banker's Algorithm is theoretically strong but rarely used directly in real operating systems.

Problem	Explanation
Maximum needs are hard to know	Processes usually cannot predict their maximum resource needs before running.
Processes change dynamically	Users log in and out, and processes start and exit continuously.
Resources can disappear	Devices may fail, so the available resource set is not perfectly fixed.
Overhead and complexity	Repeated safety checks can be costly and inconvenient.

Real-world idea

Some systems use similar heuristics. For example, a network may throttle traffic when buffer use is high, keeping enough free capacity for current users to finish.

Deadlock Prevention

1 Main Idea

Deadlock Prevention

Deadlock prevention means designing the system so that at least one of the four necessary deadlock conditions can never hold.

Deadlock avoidance is often impractical because it requires knowing future resource requests. Prevention avoids that problem by changing system rules structurally.

Prevention Rule

If any one Coffman condition is impossible, deadlock is impossible.

2 Four Conditions to Attack

Condition	Meaning
Mutual exclusion	A resource can be assigned to only one process at a time.
Hold and wait	A process can hold resources while requesting more.
No preemption	A granted resource cannot be forcibly taken away.
Circular wait	Processes form a cycle, each waiting for a resource held by the next.

3 Attacking Mutual Exclusion

Idea

If resources are never assigned exclusively, then deadlock cannot occur through those resources.

For some data, this is possible: read-only data can be shared by many processes at the same time. But many resources cannot be shared directly. Two processes writing to the same printer at the same time would produce mixed output.

Spooling

Spooling avoids direct printer sharing. Processes write output files to disk, and only the printer daemon uses the physical printer.

This removes deadlock involving the printer because the printer daemon does not request other resources. However, spooling can create another possible deadlock if disk spooling space fills up.

Spooling-space deadlock

If two processes each fill half the spooling space and both need more space to finish their output, neither can complete. The deadlock has moved from the printer to disk space.

4 Attacking Hold and Wait

Idea

Prevent processes from holding resources while waiting for more resources.

There are two common methods.

Method	Rule	Problem
Request all resources first	A process must request everything before it starts.	Processes may not know their future needs; resources are wasted.
Release before requesting	A process must release all held resources before asking for new ones.	May be inconvenient or impossible for some workflows.

Resource waste

If a process needs an input tape first, then computes for one hour, then needs an output tape and plotter, requesting everything at startup wastes the output tape and plotter during the computation hour.

Where it is used

Some mainframe batch systems require jobs to list all required resources before execution. This prevents deadlock but burdens the programmer and wastes resources.

5 Attacking No Preemption

Idea

If resources can be forcibly taken away safely, then deadlock can be broken or prevented.

This works only for resources whose state can be saved and restored safely. It is difficult for physical resources in the middle of an operation.

Resource	Preemption possible?	Reason
Memory pages	Often yes	Pages can be swapped out and restored.
Printer while printing	Usually no	Taking it away may corrupt output.
Database record lock	Usually no	The protected update must remain consistent.
Spooled printer output	More manageable	The real printer is used only by the daemon.

Virtualization limit

Spooling virtualizes the printer, but not all resources can be virtualized. Database records and operating-system tables still require locks.

6 Attacking Circular Wait

Idea

Prevent cycles by forcing all processes to request resources in a fixed global order.

One strict rule is: a process may hold only one resource at a time. If it needs another, it must release the first. This prevents circular wait, but it is often too restrictive.

A more practical method is to assign every resource type a number. Processes must request resources in increasing numerical order.

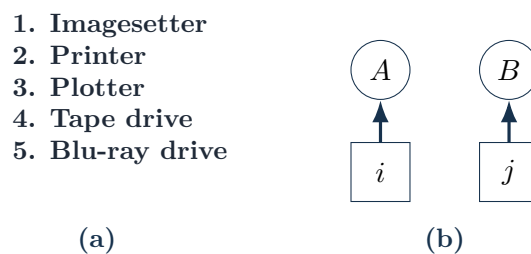


Figure: (a) Numerically ordered resources. (b) A resource graph.

Why ordering works

If every process requests resources only in increasing order, a circular wait cannot form. A cycle would require some process to request a lower-numbered resource after holding a higher-numbered one, which the rule forbids.

7 Proof Intuition for Resource Ordering

Assume every resource has a unique number. At any moment, look at the highest-numbered resource currently assigned. The process holding it cannot be waiting for any lower-numbered resource, because that is forbidden. It can only request higher-numbered resources, which are not already held by any process with a higher resource. Therefore, some process can eventually finish and release resources.

Practical difficulty

Finding one global order that works for all resources can be hard. Real systems may have many abstract resources: process-table slots, disk space, database records, locks, files, and devices.

8 Summary Table

Condition attacked	Prevention method	Main disadvantage
Mutual exclusion	Share resources where possible; use spooling for devices.	Many resources cannot be shared; spooling may shift deadlock to disk space.
Hold and wait	Require all resources at start, or release held resources before requesting more.	Wastes resources and requires advance knowledge.
No preemption	Take resources away when needed, or virtualize them.	Many resources cannot be safely preempted.
Circular wait	Number resources and require increasing request order.	A global ordering may be hard to design.

Other Deadlock Issues

1 Main Idea

Scope

Deadlock is not only a problem of ordinary hardware resources. Special cases also appear in databases, communication systems, networks, and scheduling policies.

This note covers four related topics:

- two-phase locking,
- communication deadlocks,
- livelock,
- starvation.

2 Two-Phase Locking

Definition

Two-phase locking is a database locking method where a process first obtains all required locks, then performs its updates and releases the locks.

It is useful when a transaction needs to lock several database records before updating them.

Phase	Action
Phase 1: Growing phase	The process requests locks one by one. No real update work is done.
Phase 2: Shrinking phase	After all locks are acquired, the process updates records and releases locks.

Restart rule

If a needed record is already locked during phase 1, the process releases all locks it already holds and starts phase 1 again.

Two-Phase Locking Pattern

lock everything first → perform updates → release locks

Limitation

Two-phase locking works only if the process can safely stop and restart before doing irreversible work. It is not suitable when the process has already sent network messages, updated files, or performed non-repeatable actions.

3 Communication Deadlocks

Communication Deadlock

A **communication deadlock** occurs when processes wait for messages or replies that only other waiting processes can send.

Example:

- Process A sends a request to process B .
- A blocks waiting for a reply.
- The request message is lost.
- B waits for a request.
- A waits for a reply.

This is a deadlock by the formal definition: each process is waiting for an event only another process can cause. But it is not a classical resource deadlock because no process is holding a resource wanted by the other.

Type	Cause	Typical fix
Resource deadlock	Processes compete for resources such as locks, buffers, or devices.	Ordering, prevention, detection, recovery.
Communication deadlock	Processes wait for missing messages or replies.	Timeouts and retransmission.

4 Timeouts

Timeout idea

When a process sends a message and expects a reply, it starts a timer. If the timer expires before the reply arrives, the process assumes the message was lost and sends it again.

Timeouts can break communication deadlocks because they stop a process from waiting forever.

Duplicate-message danger

If the original message was not lost but only delayed, retransmission may deliver the same request twice. This is dangerous for operations such as banking payments.

Correct communication protocols must handle duplicates, delayed replies, lost messages, and re-transmissions.

5 Resource Deadlock in a Network

Not every network deadlock is a communication deadlock. Routers also contain finite buffers, and buffers are resources.

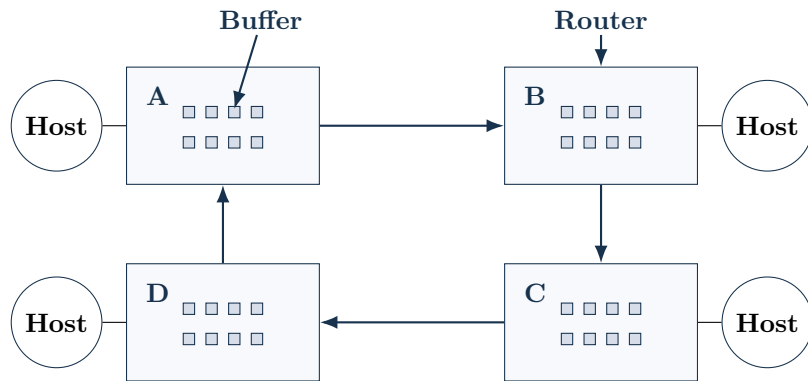


Figure: A resource deadlock in a network. Each router waits for buffer space at the next router.

Network deadlock

If all packets at *A* need to go to *B*, all packets at *B* need to go to *C*, all packets at *C* need to go to *D*, and all packets at *D* need to go to *A*, no packet can move if every next-hop buffer is full.

6 Livelock

Livelock

Livelock occurs when processes keep changing state in response to each other, but no process makes real progress.

In deadlock, processes are blocked. In livelock, processes are active, but their actions do not move the computation forward.

Polite Locking Pattern

```
process A:
    acquire resource_1
    while try_lock(resource_2) fails:
        release resource_1
        wait fixed time
        acquire resource_1

process B:
    acquire resource_2
    while try_lock(resource_1) fails:
        release resource_2
        wait fixed time
        acquire resource_2
```

Why this can livelock

If both processes release and retry at the same time, they may repeatedly collide. Neither is blocked, but neither reaches the section that uses both resources.

7 Another Livelock Example

Suppose a system has 100 process-table slots. Ten programs are running, and each needs to create 12 children.

- Each original process creates 9 children.
- The table now has $10 + 90 = 100$ processes.
- Each original process still needs 3 more children.
- Every `fork` fails because the process table is full.
- Each process waits and retries forever.

Observation

The processes are not blocked on a lock, but they still make no progress. This is livelock.

8 Starvation

Starvation

Starvation occurs when a process waits indefinitely because the resource-allocation policy keeps choosing other processes.

Example: a printer scheduler always chooses the smallest print job first. This maximizes the number of quickly completed jobs, but a huge print job may wait forever if small jobs keep arriving.

Problem	What happens	Typical fix
Starvation	A process is repeatedly postponed even though it is not deadlocked.	First-come, first-served or aging.
Deadlock	Processes wait forever for events only each other can cause.	Prevention, avoidance, detection, recovery.
Livelock	Processes keep running but repeatedly fail to make progress.	Random backoff, priority rules, or asymmetric retry behavior.

9 Starvation vs Deadlock

Key difference

A deadlocked process is blocked and cannot choose another action. A starving process may be repeatedly denied service by policy, but the system as a whole may still be active.

Starvation can be avoided with a first-come, first-served policy: the process waiting the longest receives the resource next. Eventually every request becomes the oldest request.

10 Quick Comparison

Issue	Core cause	Main idea
Two-phase locking	Need to update multiple locked records safely.	Lock first; update only after all locks are held.
Communication deadlock	Lost or missing messages cause processes to wait forever.	Use timeouts and retransmission.
Network resource deadlock	Router buffers are finite resources.	Prevent or recover from buffer cycles.
Livelock	Processes keep retrying politely in sync.	Break symmetry with random delay or ordering.
Starvation	Allocation policy repeatedly favors others.	Use fairness, FCFS, or aging.